

수천만 라인의 코드베이스, **Kiro와 함께 이해하기**

AI가 대신 이해한 사례가 아니라, 사람이 검증 가능한 질문을 할 수 있게 만든 이야기



사람이 질문한다



기계가 사실을 모은다



AI가 의미를 연결한다

발표자 소개

데이터센터와 클라우드를 넘어, AI 시대에는 통신의 시대가 온다고 믿고 있는
엔지니어 겸 대학원생입니다.

경력

LG유플러스 UCubeDAX 팀장

통신BSS 'UCube' 클라우드 운영,
사내 플랫폼 및 AI서비스 개발/운영

(전) 삼성SDS 클라우드서비스사업부

데이터센터OSS 개발, 데이터센터 보안진단, 사업부 PaaS 전문가

삼성SDS 인문학 전공자 특별 채용(SCSA)

덕분에 문서 분석, 사상적 배경, 철학 탐구에 좀 더 친숙합니다.

저서 및 기고

컨테이너 인프라 환경 구축을 위한
쿠버네티스/도커

(2021, 길벗)

readIT zine #1

(2021, 교보문고)

자격증

무면허 - (당당합니다!)

</> <https://github.com/gnu-gnu>

 <https://www.linkedin.com/in/gnu-shim/>

시스템의 이해를 이어가려 했다

시스템을 잘 아는 사람들이 짧은 기간에 팀을 떠났다.

내가 오기 전부터 쌓인 이해는 팀원들 사이에 나뉘어 있었다.

그 이해를 이어가야 했지만, 혼자 읽어 익힐 수 있는 크기는 아니었다.

AI에게 맡기면 될 줄 알았지만, 뜻대로 되지 않았다.

팀이 함께 질문하고 확인할 수 있는 형태로 바꾸기 시작했다.

그래서 먼저, 이게 얼마나 큰지 재보기로 했다.

규모를 재보니, 전략을 바꿔야 했다

전부 읽는 방식은 사람에게도 모델에게도 성립하지 않았다

전체 코드베이스

2000만 라인

수십 개 서비스 · 수백 개 저장소



한 번에 담는 컨텍스트

■ 사람이 하루에 읽는 양

사람이 전부 읽는다

하루 1,000줄씩 읽어도 수십 년

모델에 전부 넣는다

한 번에 담기는 양은 전체의 일부

컨텍스트 용량만 늘려서는 전체를 담기 어렵다. 경제적이지 않다.

나눠서 여러 번 넣는다

조각마다 판단 기준이 흔들린다

조각 사이의 관계를 놓친다

어떻게 다 읽을까



전체를 읽지 않고,
필요한 깊이까지 어떻게 내려갈까?

막힐 때마다, 믿고 있던 것을 의심했다

빠른 길로 갔다가 되돌아온 과정이다 · 첫날 만든 전체 지도는 다음 날 거의 다 틀렸다



이름과 구조로 추정했다

업무 역할 30건 이상 오판 → 약어의 뜻은 소스 주석에 이미 있었다

이름을 의심



AI를 병렬로 늘렸다

에이전트 200개, 범위·깊이가 달라 정본 불가 → 결정론적 추출로 전환

결과를 의심



검색 도구부터 붙였다

도구는 생겼는데 시작점이 없다 → 검색할 지식을 먼저 만들기로

순서를 의심

검색 기술을 고치는 대신, 검색할 지식을 먼저 만들었다

그결과, 업무 설명 확보율 3.6% → 64.6%, 분석용 빌드 성공률 78% → 100%

오늘 이야기할 것은 이 세 덩어리다

원본 근거를 구조화하고 연결한 뒤, 읽기 전용 인터페이스로 근거까지 내려간다

01 원본 근거

코드 · 구조
설정 · 빌드
업무 의미
API · 연동
데이터 접근
변화 · 운영

클래스 · 메서드 · 어노테이션
설정 · 빌드 산출물
정형 주석 · 처리 단계
엔드포인트 · 외부 호출
SQL · 엔티티 · 테이블
커밋 · 핫스팟 · 배포 단위

결정론적 추출기

원본 위치와 추출 방식을 함께 기록

VERIFIED · INFERRED · UNRESOLVED

02 관계 지도

SERVICE

공통 식별자 · 관계가 붙는 결합점

REPO · CLASS · METHOD
ENDPOINT
TABLE
TOPIC
WORKLOAD

서비스에 소속
서비스가 노출
읽고 · 쓴다
발행 · 구독
서비스를 실행

따로 뽑은 사실은 조인되어야 질문이 된다

03 읽기 전용 MCP

문서 · 의미 탐색

업무 개념 · 서비스 역할 · 플로우

코드 · 근거 탐색

코드 위치 · 원문 · 호출 관계

구조 데이터 질의

관계 · API · 데이터 접근 · 이벤트

출력 상한 · 경로 제한 · 읽기 전용

업무 질문 → 관련 문서 · 플로우 → 관계 · 정형 사실 → 코드 위치 · 호출 → 원본 근거 확인

답만 주지 않는다 · 확인된 사실 · 추론 · 미해결 상태와 원본 위치를 함께 준다

같은 코드베이스를 일곱 개의 지도로 읽었다

질문마다 필요한 사실이 달라서, 데이터의 표현을 먼저 설계했다

관점	답하려는 질문	연결할 사실
시스템	무엇이 있고 어떻게 실행되는가?	저장소·서비스·모듈·빌드·배포
진입점	처리는 어디서 시작되는가?	API·배치·스케줄러·구독자
실행 흐름	어떤 호출로 이어지는가?	클래스 의존·메서드 호출 후보
데이터·이벤트	무엇을 바꾸고 전달하는가?	SQL·CRUD·테이블·발행·구독
업무 의미	업무적으로 무엇을 하는가?	정형 주석·역할·처리 단계
핵심 로직	어디를 우선 읽어야 하는가?	분기·반복·예외·호출·SQL
변경·지식 집중	어디가 바뀌고 누가 변경했는가?	커밋·저자·변경 시점·파일 핫스팟

모든 지도에 공통으로 남긴 것

공통 식별자 + 원본 위치 + 추출 방식 + 확인·추론·미해결 상태

지식 생산 파이프라인

원본을 답으로 바꾸는 것이 아니라, 검증 가능한 중간 산출물을 쌓았다



세 역할을 섞지 않았다

기계
확인 가능한 사실 전수 추출

AI
구조화 근거 사이의 의미 연결

사람
목적·신뢰 기준·최종 판단

미확정 정보는 추측해 채우지 않고 unresolved로 남긴다

소스와 바이트코드에서 서로 다른 근거를 추출했다

목적별 결정론적 추출기를 사용하고, 추출 범위와 한계를 함께 기록했다



소스·설정

정규식·라인 스캔·본문 범위 추적
클래스·메서드 선언과 어노테이션
Javadoc·정형 주석·처리 단계
API·SQL·이벤트·설정 패턴

업무 언어와 선언을 보존
빌드 실패 코드도 분석 가능

01
10

컴파일된 바이트코드

jdeps: 클래스 의존 관계
javap -c: 호출 명령과 offset
javap -v: 어노테이션
javap -constants: 상수

빌드 성공 범위의 의존·호출 근거
offset은 실제 런타임 실행 순서가 아님

두 결과를 식별자로 연결해, 업무 의미가 붙은 호출·의존 관계를 구성했다

공통 식별자로 관점 사이를 연결했다

따로 추출한 데이터는 조인되어야 질문 가능한 지식이 된다

정규화한 식별자

저장소 / 서비스 / 클래스
메서드 / API / 데이터 객체
DB 테이블 / 이벤트 토픽 / 배포 단위



공통 ID로 관점을 연결

업무 흐름

외부 진입점 → 내부 핸들러 → 서비스 로직 → 데이터·이벤트 → 상·하류 영향



저장소



서비스



클래스·메서드

이벤트



API



핸들러



데이터 접근

원자 사실을 계층적 지식으로 집약했다

아래층은 사실, 위층은 해석이라는 경계를 계속 남겼다

근거를따라 내려가는 요약 계층



완전 자동 이해가 아니라,
근거를 따라 내려갈 수 있는 요약 계층

	시스템·운영 문서	서술
	업무 플로우	사람 판단
	서비스 간 관계	근거 연결
	서비스 역할	해석+검토
	패키지·하위 도메인	부분 해석
	클래스 역할	관계 조인
	메서드 의미	원자 사실

확정할 수 없으면 추측하지 않고 미해결 상태로 유지

병목은 검색이 아니라 초기화와 프로세스 경계였다

기존 MLX도 상주 서버에서 모델을 한 번 로드하고 재사용했다

BEFORE



Python MCP

HTTP 호출
상주 MLX 임베딩 서버
Qdrant · 별도 데이터 조회 서버

AFTER



Rust 단일 바이너리

llama.cpp 인프로세스 임베딩
Dense 직접 계산 + BM25 + RRF
DuckDB + Parquet 직접 조회

GGUF mmap·지연 로딩 · 전용 워커가 모델과 계산 컨텍스트 유지

같은 구간 비교	BEFORE	AFTER
콜드 상태 첫 검색	약 3.7초	약 0.5초
워밍 상태 임베딩 검색	HTTP 포함 약 29ms	인프로세스 약 21ms
운영 구성요소	여러 프로세스	단일 프로세스

네이티브 MCP 자체 기동 21~26ms: 모델을 아직 로드하지 않은 상태의 tools/list까지

모델 재사용을 새로 도입한 것이 아니라, 초기화 비용과 실행 경계를 제거했다

운영 구성과 외부 의존 축소

측정된 병목을 제거하고, 암호화 비용까지 포함해 확인했다

항목	이전	이후
외부 임베딩 인제스트 과금	회당 약 \$0.14	로컬 처리 · 외부 API 과금 \$0
실행 구성	Docker 2개 + Python 프로세스들	Rust 네이티브 프로세스 1개
임베딩 실행	외부 또는 별도 로컬 서버	로컬 GGUF · 인프로세스 실행
암호화 적용 비용	평문 기준	AES-256 적용 시 약 2.5배

주요 도구 호출은 암호화 적용 후에도 1초 이내

외부 API·Qdrant와 동일 조건 속도 비교는 없음
 외부 과금·자격증명·세션 만료 의존과 기동·통신·상태 관리 부담을 줄였다

AI와 파이프라인의 역할을 분리했다

AI가 사실을 만들지 않게 해야 틀렸을 때 고칠 수 있다



결정론 영역

파서 → 구조 데이터
수치·목록·관계
재실행 시 갱신
정본



해석 영역

근거 팩 → AI/사람
상위 설명 작성
확신도와 근거 기록
사람이 검토

모든 중요한 주장은 세 상태로 남긴다



확인된 사실



추론



미해결

실패를 품질 장치로 바꿨다

모든 장치는 겪은 실패에서 나왔다 · 주식 빈도를 섰더니 업무 의미가 아니라 작성자명이 1위였다

≡ 동등성 검증

컬럼형으로 바꾸는 중 내용이 조용히 빠졌다 → 추출 데이터와 같은 내용을 보존하는가

🔗 인용 검증

문서의 인용이 사라진 파일을 가리켰다 → 실제 파일·코드 위치로 돌아가는가

(×) 값 참조

서술 속 숫자가 원천 갱신 뒤 낡았다 → 값이 아니라 원천 위치와 계산 규칙을 참조하는가

↻ Blank regeneration

누적 실행은 성공, 빈 환경은 실패했다 → 산출물을 지운 환경에서 다시 만들어지는가

🔗 의존성 검증

남아 있던 중간 산출물이 순서 결함을 가렸다 → 단계별 producer/consumer 순서가 명시되었는가

🔍 Negative test

검증기가 진짜 잡는지 알 수 없었다 → 고의 오류·잘못된 인용을 실패로 잡는가

❓ Unresolved 유지

확인 못 한 칸을 그럴듯하게 채웠다 → 추정으로 채우지 않는가

결과가 한 번 생성됐다는 사실보다, 동일한 입력에서 다시 생성되고 오류를 의도적으로 잡아낼 수 있다는 사실이 더 중요했다

질문 하나가 답에 도달하는 경로

전체를 읽지 않고 질문에 필요한 깊이까지만 내려간다

[예시] “가입 처리 흐름에서 청구 연동은 어디서 일어나는가?”



업무 개념 검색

플로우·서비스 후보



구조 데이터 확인

API·데이터·이벤트



코드 위치 탐색

핸들러·로직 후보



호출 관계 확인

상·하류 영향



원문 조회

필요한 코드만

읽는 양

업무 범위에서 필요한 원문으로 점차 좁혀간다

근거가 부족하면 “미해결”이라고 답하는 것이 정상 동작

분산 트랜잭션이 실패하면 무엇이 남는가?

장애 대응에서 가장 자주 받는 질문 · 익명화한 가정 예시

실패 상황을 가정한 예시 · 실제 실행 순서와 완료 여부는 별도 확인

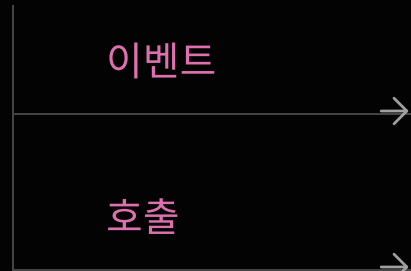
((o)) 요청



 서비스 A



 DB 변경①



이벤트

 서비스 C



 DB 변경②

호출

 서비스 B



실패

fallback·retry·rollback·보상 확인 필요

CPG의 관점 결합

제어 흐름 + 데이터 의존

이번 작업에서는

서비스 호출 + 데이터 쓰기 + 이벤트

실패 후 남을 수 있는 데이터 변경과 후속 서비스의 side-effect 후보를 찾는다

자산은 문서가 아니라 다시 만드는 과정이다

문서보다 재생성 파이프라인과 외부화된 작업 기억이 오래 간다

-  원본 코드와 설정
-  구조화된 사실 데이터
-  데이터 계약과 생성 이력
-  서비스·업무 단위 근거 팩
-  사람이 관리하는 해석
-  서비스·플로우·운영 문서
-  검증과 신선도 정보
-  재생성 절차와 변경 기록



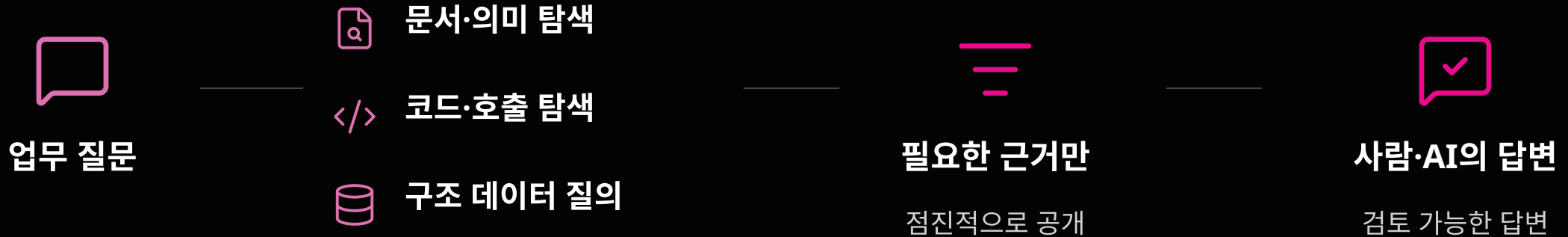
세션 밖 기억

근거 팩
서술 조각
체크포인트
작업 기록

다음 작업은 이전 대화를 기억하는 대신 산출물을 다시 읽고 이어간다

MCP는 마지막 인터페이스였다

열어줄 지식 구조를 만든 뒤, 읽기 전용으로 사람과 AI에게 제공했다



보안은 세 층으로 설계했다



데이터 보호

로컬 처리 · AES-256 암호화 · 키 분리



접근 통제

읽기 전용 SQL · 경로 제한 · 원본 보호



AI 사용 안전성

출력 제한 · 근거 제시 · 실패 격리

운영 시스템이나 원본 코드를 직접 변경하는 도구가 아니다

질문의 추상도가 다르면, 들어가는 층도 달라야 한다

이론을 구현한 것이 아니라, 문제를 풀다가 이론과 수렴했다 · RAPTOR(Sarathi 외, ICLR 2024)와의 대응은 사후 해석이다

설명 집약 ↑

시스템·운영 관점
업무 플로우
서비스 역할
패키지·하위 도메인
클래스 역할
메서드 의미
원본 코드·SQL·호출·위치

↓ 근거 확인

- ← 시스템은 어떻게 구성돼 있나
- ← 가입은 어떤 서비스를 거치나
- ← 이 서비스는 무엇을 담당하나
- ← 이 메서드는 어떤 데이터를 바꾸나
- ← 파일·라인 단위 검증 지점

RAPTOR

임베딩 군집과 재귀 요약으로 트리를 구성

이번 작업

소프트웨어 구조를 따라 집약 구조 데이터와 여러 추상도의 문서 섹션을 검색

공통점: 질문에 맞는 설명 수준 탐색
차이: 군집·트리 알고리즘은 미구현

상위 설명에서 시작해, 검증 가능한 원본 근거까지 내려갈 수 있게 만들었다

논문: arXiv:2401.18059 · NarrativeQA·QASPER·QuALITY 개선 보고. 본 시스템의 성과 수치와 구분

CODEBASE / KNOWLEDGE ENGINEERING

같은 문장도, 어느 서비스의 어느 절인지 알아야 한다

먼저 문맥을 함께 저장했고, 나중에 Contextual Retrieval이 같은 문제를 다룬 걸 알았다 · Anthropic, 2024

본문만 저장하면 생기는 문제

“상태를 변경한다”
process() / update()

어느 업무의 어느 단계인지 알 수 없다
4자 약어 식별자가 의미 공간에서 혼동되어
다른 서비스의 결과가 섞였다



문맥을 함께 저장한 검색 단위

서비스 범위	가입 처리 서비스
문서 종류	업무 플로우
제목 계층	개통 처리 > 상태 갱신 > 실패 처리
본문	“상태를 변경한다 ...”
근거	파일 경로 : 라인

서비스·문서 종류로 검색 범위를 제한한다

Anthropic

청크마다 LLM이 설명 문맥을 만들어 색인에 추가

이번 작업

구조적 문맥을 보존하고 범위를 필터링 · LLM 문맥 문장 생성은 미사용

청크를 고립시키지 않고, 어느 문서의 어느 절인지를 함께 저장했다

질문이 달라지면 펼치는 지도가 달라진다

먼저 질문별 지도를 만들었고, 나중에 Aider repo map이 같은 선택을 한 걸 알았다

관점별 지도 의미 · 호출 · 데이터 · 연동 · 변경

-  장애가 어디까지 번지나 → 호출 · 이벤트 · 공유 데이터
-  이 코드를 바꾸면 → 핵심 로직 · 의존 관계 · 변경 이력
-  이 업무는 무엇을 하나 → 업무 의미 · 진입점 · 처리 단계

비교 축	Aider repo map	이번 작업
지도 재료	Tree-sitter 심볼·참조	소스 패턴 + 바이트코드 의존·호출 + 정형 주석
선택 기준	의존 그래프 중요도 랭킹	질문별 지도 선택 · 로직 밀도·변경 집중도 참고
컨텍스트 제한	토큰 예산 안의 핵심 심볼	결과 크기 상한 + 필요한 원문만 단계적 조회

Aider repo map과의 연결: 원문보다 지도를 먼저 제공한다

출처: Aider 공식 repo map 문서 · 동일 랭킹 미구현, 토큰 절감률·결함 예측 성능 미측정

세션은 끝나지만, 산출물은 남는다

먼저 파일로 남겼고, 나중에 재귀 요약·Refine과 연결됐다 · Wu 외, 2021

여러 세션에 걸친 분석에서, 대화 맥락이 자동으로 이어진다고 가정할 수 없었다



파일로 남는 작업 기억

구조 데이터 · 근거 팩 · 서술 조각 · 체크포인트 · 작업 기록 · 미해결 목록

구분	연구·기법	이번 작업
집약·보강	재귀 요약: 부분을 재요약 Refine: 다음 근거로 순차 보강	메서드·서비스 근거를 집약 새 사실·미해결 항목 반영 후 재검증
모델 학습	재귀 요약 논문: 사람 피드백으로 학습	모델 학습 없음 · 논문 지표의 요약 품질 미측정

연속성을 대화 기억이 아니라 파일에 맡겼다 · Kiro 크레딧 7,500도 산출물에 썼다

향후 과제: 실패 영향 후보의 정밀 검증

정적 관계와 실제 실행 증거를 결합해 잔존 변경·중복 처리·보상을 확인



1. 요청과 side-effect의 직접 연결

Endpoint → Handler, Method → Table·Event·외부 호출
서비스 수준 후보를 요청·메서드 수준으로 좁힌다



2. 실패 처리와 데이터 흐름 분석

표현식 AST·CFG·변수 정의·사용·Data Flow 구축
예외 전파와 catch·retry·fallback·보상 경로를 연결한다



3. 실행·완료 증거와 대조

트레이스·트랜잭션·이벤트 처리 기록으로 commit·소비 여부 확인
재시도 중복·먹등성·보상 성공은 장애 주입 시나리오로 검증한다

목표 질문

“실패 뒤 어떤 변경이 남았고, 재시도·보상이 이를 어떻게 처리했는가?”

말하지 않기로 한 것들

- 정적 분석만으로 리플렉션·동적 프록시·런타임 문자열 조립을 완전히 복원하지 않는다
- 소스의 호출 관계를 실제 런타임 순서로 단정하지 않는다
- 모든 업무 의미가 도메인 담당자의 검수를 받은 것은 아니다
- 검색 결과는 확률적이며 원본 근거 확인이 필요하다
- 측정하지 않은 ROI나 생산성 향상 수치를 주장하지 않는다
- 코드베이스 전체를 완전히 이해했다고 말하지 않는다

남길 문장

**기계는 사실을 전수 추출하고,
AI는 근거 사이의 의미를 연결하며,
사람은 목적과 판단 기준을 제공한다.**

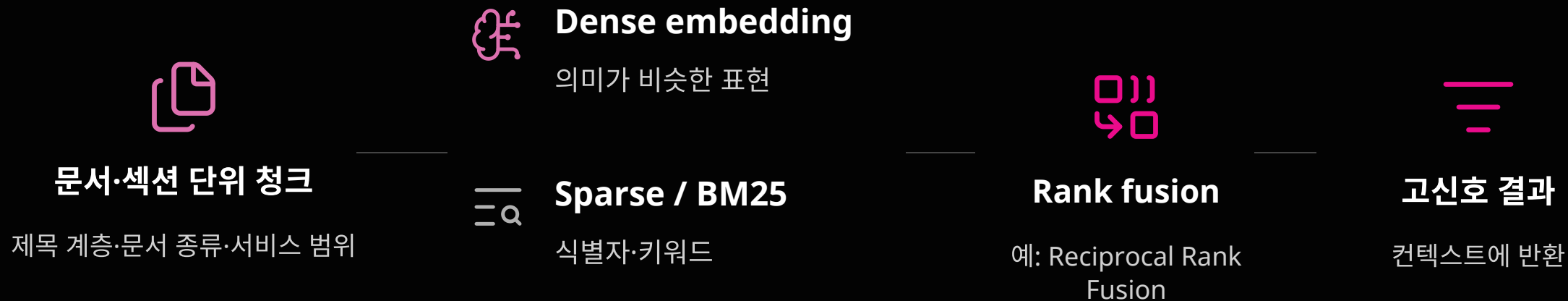
We are hiring

- 기술이 비즈니스를 가속화한다고 믿는
- 결정에는 측정이 따라오는
- 도구를 늘리기 전에 기본기로 돌아가는
- 가보지 않은 길에 두려움 없이, 늘 함께 도전하는
- 성공에는 만족하지 않으며 실패는 기록하는
- 엔터프라이즈 비즈니스가 가져야 하는 책임감을 이해하는

개발, AI, 인프라, DevOps/SRE, 보안, Project Management
어떤 역량이든 저희는 필요한 일을 만들어가고 있습니다.

구조를 보존하는 하이브리드 검색

기존 Python/Qdrant에서도 Dense·BM25·RRF와 서비스 메타데이터 필터를 사용했다



구조적 contextualization

긴 문서를 통째로 임베딩하지 않고, 청크마다 상위 제목·카테고리·서비스 범위를 보존한다
 서비스·문서 종류 필터로 오검색을 줄이고, 구조 데이터·코드 위치·호출 관계로 내려간다
 내부 데이터는 외부 임베딩/API로 보내지 않고 로컬에서 처리한다

측정에 따른 세 단계 최적화

M2 Pro 32GB 실측 기록 · 단계마다 바꾼 대상과 비교 기준이 다르다



1. 임베딩 로컬화

외부 임베딩 → 로컬 MLX

외부 API 과금·자격증명 의존 제거



2. 실행 구조 통합

Python·Docker·Qdrant → Rust

Docker 2개와 Python 프로세스들을
네이티브 프로세스 1개로 통합



3. 세부 최적화

런타임·저장·검색·암호화

지연 로딩·전용 워커·직접 탐색
Parquet·양자화·암호화 검증

측정해 보니 검색 연산보다 기동과 구성요소의 비용이 더 컸다

현재 규모에는 별도 벡터 DB가 불필요했다

Dense + BM25 + RRF와 서비스 메타데이터 필터는 레거시에도 있었다

직접 전수 내적

0.057 ms

약 2천 개 벡터 · 9.3MB 메모리 상주

탐색 대상	직접 탐색 실측
현재 약 2천 개	0.057ms
10배 합성 데이터	1.65ms
100배 합성 데이터	19.1ms

검색 기능을 이식하고 결과를 검증

Python BM25를 Rust로 이식 · Qdrant의 dense/sparse/RRF를 직접 계산으로 대체
Python 결과와 골든셋 29/29 일치

전수 내적 0.057ms로 현재 규모에 충분함을 확인 · Qdrant 동일 조건 지연시간은 미측정

파일 형식 최적화와 DuckDB 통합

CSV → Parquet 수치는 Python → Rust 성능 비교가 아니다

측정 항목	CSV	Parquet + zstd	변화
데이터 크기	473MB	34MB	약 14배 작음
집계 시간	65ms	3ms	약 22배 빠름



DuckDB·Parquet는 Python 구조에도 존재

Rust 전환: DuckDB 정적 링크 · 별도 API 서버와 컨테이너 제거
필요한 Parquet만 on-demand view로 등록 · 별도 DB 인제스트 제거

반복 조회할 의미·관계는 사전 계산하고, 상세 원문은 필요할 때 조회

8비트 GGUF 배포와 임베딩 호환성 검증

배포 크기의 감소와 추론 속도의 변화를 분리해서 본다

bf16

1,198 MB



GGUF Q8_0 · 8비트 양자화

610 MB

약 49% 축소 · 1.96배 작아짐

호환성 비교	코사인 유사도
MLX bf16 vs llama.cpp Q8_0	중앙값 0.983
MLX bf16 vs llama.cpp f16	0.9835
llama.cpp 인프로세스 vs CLI	0.999999~1.000000

bf16에서도 차이가 비슷해 양자화를 주원인으로 단정하지 않았다

llama.cpp로 코퍼스 벡터 전체를 재생성 · 양자화 단독 속도 개선 수치는 없음